associations that are needed further up the stack (in the service or the view layer) must be loaded before the Hibernate `Session` is closed.

Unfortunately, there are bad ways to deal with the `LazyInitializationException` too. One quick fix would be to change the association in question to `FetchType.EAGER`. While this would work for the current business use case, the `FetchType.EAGER` is going to affect all other queries where the root entity of this association is fetched.

The fetching strategy is a query time responsibility, and each query should only fetch just as much data that is needed by the current business use case. On the other hand, `FetchType.EAGER` is a mapping time decision that is taken outside the business logic context where the association is meant to be used.

There is also the *Open Session in View* anti-pattern that is sometimes proposed as a solution for the `LazyInitializationException`.

14.3.3.2.4 The Open Session in View Anti-Pattern

Open Session in View is an architectural pattern that proposes to hold the Persistence Context open throughout the whole web request. This way, if the service layer fetched an entity without fully initializing all its associations further needed by the UI, then the view layer could silently trigger a Proxy initialization on demand.

Spring framework comes with a `javax.servlet.Filter`[1] implementation of the Open Session in View pattern. The `OpenSessionInViewFilter` gets a `Session` from the underlying `SessionFactory` and registers it in a `ThreadLocal` storage where the `HibernateTransactionManager` can also locate it. This service layer is still responsible for managing the actual JDBC or JTA transaction, but the `Session` is no longer closed by the `HibernateTransactionManager`[2].

[1] https://docs.spring.io/spring/docs/current/javadoc-api/org/springframework/orm/hibernate5/support/OpenSessionInViewFilter.html

[2] https://docs.spring.io/spring/docs/current/javadoc-api/org/springframework/orm/hibernate5/HibernateTransactionManager.html

To visualize the whole process, consider the following sequence diagram:

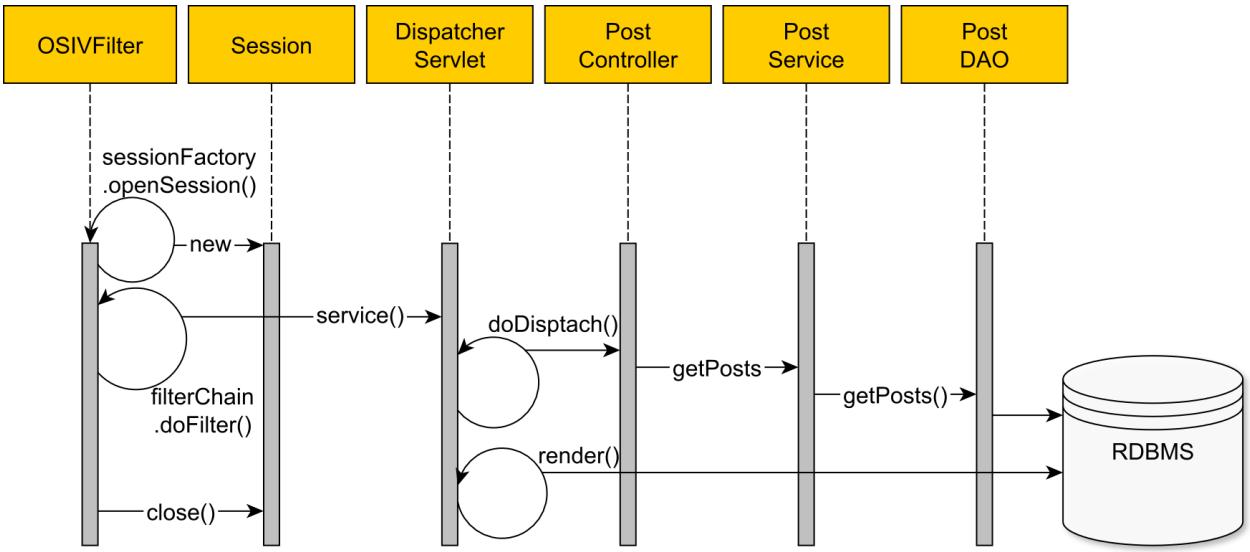

Figure 14.3: Open Session in View lifecycle

- The `OpenSessionInViewFilter` calls the `openSession` method of the underlying `SessionFactory` and obtains a new `Session`.
- The `Session` is bound to the `TransactionSynchronizationManager`[3].
- The `OpenSessionInViewFilter` calls the `doFilter` of the `javax.servlet.FilterChain` object reference and the request is further processed
- The `DispatcherServlet`[4] is called, and it routes the HTTP request to the underlying `PostController`.
- The `PostController` calls the `PostService` to get a list of `Post` entities.
- The `PostService` opens a new transaction, and the `HibernateTransactionManager` reuses the same `Session` that was opened by the `OpenSessionInViewFilter`.
- The `PostDAO` fetches the list of `Post` entities without initializing any lazy association.
- The `PostService` commits the underlying transaction, but the `Session` is not closed because it was opened externally.
- The `DispatcherServlet` starts rendering the UI, which, in turn, navigates the lazy associations and triggers their initialization.
- The `OpenSessionInViewFilter` can close the `Session`, and the underlying database connection is released as well.

At a first glance, this might not look like a terrible thing to do, but, once you view it from a database perspective, a series of flaws start to become more obvious.

[3] http://docs.spring.io/spring/docs/current/javadoc-api/org/springframework/transaction/support/TransactionSynchronizationManager.html

[4] http://docs.spring.io/spring/docs/current/javadoc-api/org/springframework/web/servlet/DispatcherServlet.html

The service layer opens and closes a database transaction, but afterward, there is no explicit transaction going on. For this reason, every additional statement issued from the UI rendering phase is executed in auto-commit mode. Auto-commit puts pressure on the database server because each statement must flush the transaction log to disk, therefore causing a lot of I/O traffic on the database side. One optimization would be to mark the `Connection` as read-only which would allow the database server to avoid writing to the transaction log.

There is no separation of concerns anymore because statements are generated both by the service layer and by the UI rendering process. Writing integration tests that assert the number of statements being generated requires going through all layers (web, service, DAO), while having the application deployed on a web container. Even when using an in-memory database (e.g. HSQLDB) and a lightweight web server (e.g. Jetty), these integration tests are going to be slower to execute than if layers were separated and the back-end integration tests used the database, while the front-end integration tests were mocking the service layer altogether.

The UI layer is limited to navigating associations which can, in turn, trigger N+1 query problems, as previously explained. Although Hibernate offers `@BatchSize`[5] for fetching associations in batches, and `FetchMode.SUBSELECT`[6] to cope with this scenario, the annotations are affecting the default fetch plan, so they get applied to every business use case. For this reason, a data access layer query is much more suitable because it can be tailored for the current use case data fetch requirements.

Last but not least, the database connection is held throughout the UI rendering phase which increases connection lease time and limits the overall transaction throughput due to congestion on the database connection pool. The more the connection is held, the more other concurrent requests are going to wait to get a connection from the pool.

The Open Session in View is a solution to a problem that should not exist in the first place, and the most likely root cause is relying exclusively on entity fetching.

If the UI layer only needs a view of the underlying data, then the data access layer is going to perform much better with a DTO projection. A DTO projection forces the application developer to fetch just the required data set and is not susceptible to `LazyInitializationException(s)`.

This way, the separation of concerns is no longer compromised, and performance optimizations can be applied at the data access layer since all statements are confined to the boundaries of the currently executing transaction.

[5] https://docs.jboss.org/hibernate/orm/current/javadocs/org/hibernate/annotations/BatchSize.html
[6] https://docs.jboss.org/hibernate/orm/current/javadocs/org/hibernate/annotations/FetchMode.html#SUBSELECT

14.3.3.2.5 Temporary Session Lazy Loading Anti-Pattern

Analogous to the Open Session in View, Hibernate offers the `hibernate.enable_lazy_load_no_trans` configuration property which allows an uninitialized lazy association to be loaded outside of the context of its original Persistence Context.

```
<property name="hibernate.enable_lazy_load_no_trans" value="true"/>
```

With this configuration property in place, the following code snippets can be executed without throwing any `LazyInitializationException`:

```
List<PostComment> comments = null;

EntityManager entityManager = null;
EntityTransaction transaction = null;
try {
    entityManager = entityManagerFactory().createEntityManager();
    transaction = entityManager.getTransaction();
    transaction.begin();

    comments = entityManager.createQuery(
        "select pc " +
        "from PostComment pc " +
        "where pc.review = :review", PostComment.class)
    .setParameter("review", review)
    .getResultList();

    transaction.commit();
} catch (Throwable e) {
    if ( transaction != null && transaction.isActive())
        transaction.rollback();
    throw e;
} finally {
    if (entityManager != null) {
        entityManager.close();
    }
}
for(PostComment comment : comments) {
    LOGGER.info("The post title is '{}'", comment.getPost().getTitle());
}
```

Behind the scenes, a temporary `Session` is opened just for initializing every `post` association. Every temporary `Session` implies acquiring a new database connection, as well as a new database transaction.

The more associations being loaded lazily, the more additional connections are going to be requested which puts pressure on the underlying connection pool. Each association being loaded in a new transaction, the transaction log is forced to flush after each association initialization.

Just like Open Session in View, the `hibernate.enable_lazy_load_no_trans` configuration property is an anti-pattern as well because it only treats the symptoms and does not solve the actual cause of the `LazyInitializationException`.

By properly initializing all lazy associations prior to closing the initial Persistence Context, and switching to DTO projections where entities are not even necessary, the `LazyInitializationException` is prevented in a much more efficient way.

14.3.3.3 Associations and pagination

As previously explained, paginating result sets has many benefits, from lowering the response time to ensuring that the application works with the ever increasing data sets. Also, fetching a collection with the `join fetch` JPQL directive can prevent N+1 query problems and `LazyInitializationException(s)` as well. Unfortunately, mixing collection fetching and pagination does not work very well together.

Collections must always be fetched fully because otherwise the collection size might not be consistent with the number of child entries associated with a given parent. On the other hand, SQL pagination can truncate the collection before returning all child records, therefore breaking the aforementioned consistency guarantee.

To visualize this process, the following entity query is going to load a list of `Post` entities, filtered by their `title`, and also, fetch all comments associated with a given `Post` record.

When specifying a `maxResults` restriction:

```
List<Post> posts = entityManager.createQuery(
    "select p " +
    "from Post p " +
    "left join fetch p.comments " +
    "where p.title like :title " +
    "order by p.id", Post.class)
.setParameter("title", titlePattern)
.setMaxResults(50)
.getResultList();
```

Hibernate issues a warning message saying that pagination is done in memory, and the SQL query shows no sign of limiting the result set:

```
WARN - firstResult/maxResults specified with collection fetch;
       applying in memory!

SELECT p.id AS id1_0_0_, pc.id AS id1_1_1_, p.title AS title2_0_0_,
       pc.post_id AS post_id3_1_1_, pc.review AS review2_1_1_,
       pc.post_id AS post_id3_1_0__, pc.id AS id1_1_0__
FROM   post p
LEFT OUTER JOIN post_comment pc ON p.id = pc.post_id
WHERE  p.title LIKE 'high-performance%'
ORDER BY p.id
```

So, Hibernate fetches the whole result set, and then it limits the number of root entities according to the `maxResults` query attribute value.

Compared to SQL-level pagination, entity query result set size restriction is not very efficient, causing the database to fetch the whole result set.

Entity queries vs DTO projections

By now, it is obvious that entity queries, although useful in certain scenarios, are not a universal solution to fetching data from a relational database. It can be detrimental to application performance to rely only on entity queries exclusively. As a rule of thumb, entity queries should be used when there is a need to modify the currently selected entities.

For read-only views, DTO projections can be more efficient because there are fewer columns being selected, and the queries can be paginated at the SQL-level. While the entity query language (JPQL and HQL) offers a wide range of filtering criteria, a native SQL query can take advantage of the underlying relational database querying capabilities.

JPQL/HQL and SQL queries are complementary solutions, both having a place in an enterprise system developer's toolkit.

14.3.4 Attribute lazy fetching

When fetching an entity, all attributes are going to be loaded as well. This is because every entity attribute is implicitly marked with the `@Basic`[7] annotation whose default fetch policy is `FetchType.EAGER`.

However, the attribute fetch strategy can be set to `FetchType.LAZY`, in which case the entity attribute is loaded with a secondary select statement upon being accessed for the first time.

```
@Basic(fetch = FetchType.LAZY)
```

This configuration alone is not sufficient because Hibernate requires bytecode instrumentation to intercept the attribute access request and issue the secondary select statement on demand.

When using the Maven bytecode enhancement plugin, the `enableLazyInitialization` configuration property must be set to `true` as illustrated in the following example:

```
<plugin>
    <groupId>org.hibernate.orm.tooling</groupId>
    <artifactId>hibernate-enhance-maven-plugin</artifactId>
    <version>${hibernate.version}</version>
    <executions>
        <execution>
            <configuration>
                <failOnError>true</failOnError>
                <enableLazyInitialization>true</enableLazyInitialization>
            </configuration>
            <goals>
                <goal>enhance</goal>
            </goals>
        </execution>
    </executions>
</plugin>
```

With this configuration in place, all JPA entity classes are going to be instrumented with lazy attribute fetching. This process takes place at build time, right after entity classes are compiled from their associated source files.

The attribute lazy fetching mechanism is very useful when dealing with column types that store large amounts of data (e.g. `BLOB`, `CLOB`, `VARBINARY`). This way, the entity can be fetched without automatically loading data from the underlying large column types, therefore improving performance.

To demonstrate how attribute lazy fetching works, the following example is going to use an `Attachment` entity which can store any media type (e.g. PNG, PDF, MPEG).

[7] http://docs.oracle.com/javaee/7/api/javax/persistence/Basic.html#fetch--

```
@Entity @Table(name = "attachment")
public class Attachment {

    @Id
    @GeneratedValue
    private Long id;

    private String name;

    @Enumerated
    @Column(name = "media_type")
    private MediaType mediaType;

    @Lob
    @Basic(fetch = FetchType.LAZY)
    private byte[] content;

    //Getters and setters omitted for brevity
}
```

Attributes such as the entity identifier, the name or the media type are to be fetched eagerly on every entity load. On the other hand, the media file content should be fetched lazily, only when being accessed by the application code.

After the `Attachment` entity is instrumented, the class bytecode is changed as follows:

```
@Transient
private transient PersistentAttributeInterceptor
    $$_hibernate_attributeInterceptor;

public byte[] getContent() {
    return $$_hibernate_read_content();
}

public byte[] $$_hibernate_read_content() {
    if ($$_hibernate_attributeInterceptor != null) {
        this.content = ((byte[]) $$_hibernate_attributeInterceptor
            .readObject(this, "content", this.content));
    }
    return this.content;
}
```

The `content` attribute fetching is done by the `PersistentAttributeInterceptor` object reference, therefore providing a way to load the underlying `BLOB` column only when the getter is called for the first time.

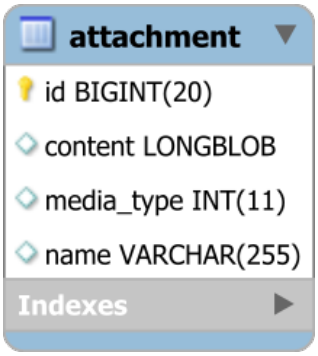

Figure 14.4: The attachment database table

When executing the following test case:

```
Attachment book = entityManager.find(Attachment.class, bookId);

LOGGER.debug("Fetched book: {}", book.getName());

assertArrayEquals(Files.readAllBytes(bookFilePath), book.getContent());
```

Hibernate generates the following SQL queries:

```
SELECT a.id AS id1_0_0_,
       a.media_type AS media_ty3_0_0_,
       a.name AS name4_0_0_
FROM   attachment a
WHERE  a.id = 1

-- Fetched book: High-Performance Java Persistence

SELECT a.content AS content2_0_
FROM   attachment a
WHERE  a.id = 1
```

Because it is marked with the `FetchType.LAZY` annotation and lazy fetching bytecode enhancement is enabled, the `content` column is not fetched along with all the other columns that initialize the `Attachment` entity. Only when the data access layer tries to access the `content` attribute, Hibernate issues a secondary select to load this attribute as well.

Just like `FetchType.LAZY` associations, this technique is prone to N+1 query problems, so caution is advised. One slight disadvantage of the bytecode enhancement mechanism is that all entity attributes, not just the ones marked with the `FetchType.LAZY` annotation, are going to be transformed, as previously illustrated.

14.3.5 Fetching subentities

Another approach to avoid loading table columns that are rather large is to map multiple subentities to the same database table.

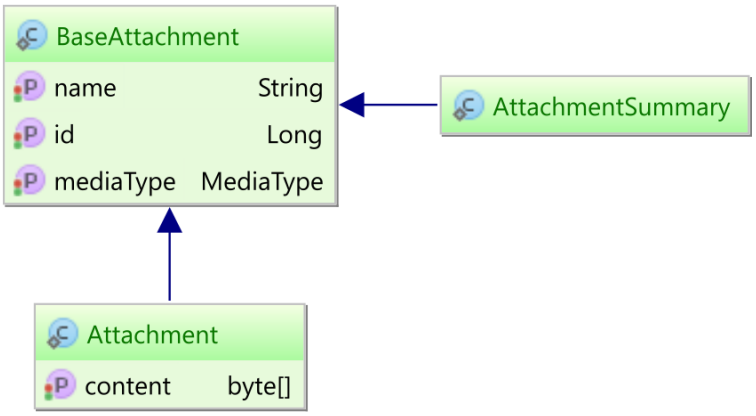

Figure 14.5: Attachment and AttachmentSummary entities

Both the `Attachment` entity and the `AttachmentSummary` subentity inherit all common attributes from a `BaseAttachment` superclass.

```
@MappedSuperclass
public class BaseAttachment {

    @Id
    @GeneratedValue
    private Long id;

    private String name;

    @Enumerated
    @Column(name = "media_type")
    private MediaType mediaType;

    //Getters and setters omitted for brevity
}
```

While `AttachmentSummary` extends `BaseAttachment` without declaring any new attribute:

```
@Entity @Table(name = "attachment")
public class AttachmentSummary extends BaseAttachment {}
```

The `Attachment` entity inherits all the base attributes from the `BaseAttachment` superclass and maps the `content` column as well.

```
@Entity @Table(name = "attachment")
public class Attachment extends BaseAttachment {

    @Lob
    private byte[] content;

    //Getters and setters omitted for brevity
}
```

When fetching the `AttachmentSummary` subentity:

```
AttachmentSummary bookSummary = entityManager.find(
    AttachmentSummary.class, bookId);
```

The generated SQL statement is not going to fetch the `content` column:

```
SELECT a.id as id1_0_0_, a.media_type as media_ty2_0_0_, a.name as name3_0_0_
FROM attachment a
WHERE  a.id = 1
```

However, when fetching the `Attachment` entity:

```
Attachment book = entityManager.find(Attachment.class, bookId);
```

Hibernate is going to fetch all columns from the underlying database table:

```
SELECT a.id as id1_0_0_, a.media_type as media_ty2_0_0_,
       a.name as name3_0_0_, a.content as content4_0_0_
FROM attachment a
WHERE  a.id = 1
```

When it comes to reading data, subentities are very similar to DTO projections. However, unlike DTO projections, subentities can track state changes and propagate them to the database.

14.4 Entity reference deduplication

Considering that the `Post` comment has a bidirectional `@OneToMany` association with a `PostComment` entity, and the database contains the following entities:

```
Post post = new Post();
post.setId(1L);
post.setTitle("High-Performance Java Persistence");
post.addComment(new PostComment("Excellent!"));
post.addComment(new PostComment("Great!"));

entityManager.persist(post);
```

When fetching a `Post` entity along with all its `PostComment` child entries:

```
List<Post> posts = entityManager.createQuery(
    "select p " +
    "from Post p " +
    "left join fetch p.comments " +
    "where p.title = :title", Post.class)
.setParameter("title", "High-Performance Java Persistence")
.getResultList();

LOGGER.info("Fetched {} post entities: {}", posts.size(), posts);
```

Hibernate generates the following output:

```
SELECT p.id AS id1_0_0_ , pc.id AS id1_1_1_ , p.title AS title2_0_0_ ,
       pc.post_id AS post_id3_1_1_, pc.review AS review2_1_1_
FROM   post p
LEFT OUTER JOIN post_comment pc ON p.id = pc.post_id
WHERE  p.title = 'High-Performance Java Persistence'

-- Fetched 2 post entities: [
    Post{id=1, title='High-Performance Java Persistence'},
    Post{id=1, title='High-Performance Java Persistence'}]
```

Because the underlying SQL query result set size is given by the number of `post_comment` rows, and the `post` data is duplicated for each associated `post_comment` entry, Hibernate is going to return 2 `Post` entity references.

Because the Persistence Context guarantees application-level repeatable reads, the `posts` list contains two references to the same `Post` entity object. To enable entity reference deduplication, JPA and Hibernate provide the `distinct` keyword.

Therefore, when adding `distinct` to the previous entity query:

```
List<Post> posts = entityManager.createQuery(
    "select distinct p " +
    "from Post p " +
    "left join fetch p.comments " +
    "where p.title = :title", Post.class)
.setParameter("title", "High-Performance Java Persistence")
.getResultList();
```

Hibernate generates the following output:

```
SELECT DISTINCT
       p.id AS id1_0_0_ , pc.id AS id1_1_1_ , p.title AS title2_0_0_ ,
       pc.post_id AS post_id3_1_1_, pc.review AS review2_1_1_
FROM   post p
LEFT OUTER JOIN post_comment pc ON p.id = pc.post_id
WHERE  p.title = 'High-Performance Java Persistence'

-- Fetched 1 post entities: [
    Post{id=1, title='High-Performance Java Persistence'}]
```

So, the duplicated entries have been removed from the result set, but the `DISTINCT` keyword was passed to the underlying SQL query. While this would be beneficial for scalar queries, for entity queries, this can affect the query execution plan.

When executing the query above that with the `DISTINCT` keyword on PostgreSQL, the following execution plan is obtained:

```
HashAggregate
  Group Key: p.id, pc.id, p.title, pc.post_id, pc.review
  -> Hash Right Join
    Hash Cond: (pc.post_id = p.id)
    -> Seq Scan on post_comment pc
    -> Hash
         -> Seq Scan on post p
              Filter: (title = 'High-Performance Java Persistence')
```

The `HashAggregate` is going to execute a sort the result set so that duplicate entries can be removed much faster. In this particular use case, this extra sorting phase is completely redundant because there are no duplicate entries to be removed. Therefore, the overall response time is going to be increased unnecessarily.

For this reason, Hibernate 5.2.2 adds an optimization via the `DISTINCT_PASS_THROUGH` query hint. When providing this query hint, and rerunning the previous entity query:

```
List<Post> posts = entityManager.createQuery(
    "select distinct p " +
    "from Post p " +
    "left join fetch p.comments " +
    "where p.title = :title", Post.class)
.setParameter("title", "High-Performance Java Persistence")
.setHint(QueryHints.HINT_PASS_DISTINCT_THROUGH, false)
.getResultList();
```

Hibernate is going to generate the following output:

```
SELECT p.id AS id1_0_0_ , pc.id AS id1_1_1_ , p.title AS title2_0_0_ ,
       pc.post_id AS post_id3_1_1_, pc.review AS review2_1_1_
FROM   post p
LEFT OUTER JOIN post_comment pc ON p.id = pc.post_id
WHERE  p.title = 'High-Performance Java Persistence'

-- Fetched 1 post entities: [
    Post{id=1, title='High-Performance Java Persistence'}]
```

So, the entity references have been deduplicated while the `distinct` JPA keyword was not passed through the underlying SQL statement. This time, the PostgreSQL execution plan looks as follows:

```
Hash Right Join
  Hash Cond: (pc.post_id = p.id)
  ->  Seq Scan on post_comment pc
  ->  Hash
        -> Seq Scan on post p
             Filter: (title = 'High-Performance Java Persistence')
```

As illustrated by the execution plan above, there is no `HashAggregate` step this time. Therefore, the unnecessary sorting phase is skipped, and the query execution is going to be faster.

14.5 Query plan cache

There are two types of entity queries: dynamic and named queries. For dynamic queries, the `EntityManager` offers the `createQuery` method, while for named queries, there is the `createNamedQuery` alternative. There is no obvious performance gain for using named queries over dynamic ones because, behind the scenes, a named query is able to cache only its definition (e.g. `NamedQueryDefinition`), and the actual query plan cache is available for both dynamic and named queries.

Every query must be compiled prior to being executed, and, because this process might be resource intensive, Hibernate provides a `QueryPlanCache` for this purpose. For entity queries, the query `String` representation is parsed into an Abstract Syntax Tree. For native queries, the phase extracts information about the named parameters and query return type.

The query plan cache is shared by entity and native queries, and its size is controlled by the following configuration property:

```
<property name="hibernate.query.plan_cache_max_size" value="2048"/>
```

By default, the `QueryPlanCache` stores 2048 plans which is sufficient for many small and medium-sized enterprise applications.

For native queries, the `QueryPlanCache` stores also the `ParameterMetadata` which holds info about parameter name, position, and associated Hibernate type. The `ParameterMetadata` cache is controlled via the following configuration property:

```
<property name="hibernate.query.plan_parameter_metadata_max_size" value="128"/>
```

If the application executes more queries than the `QueryPlanCache` can hold, there is going to be a performance penalty due to query compilation.

Next, we are going to run a test which executes only two queries while varying the `QueryPlanCache` and the `ParameterMetadata` cache size from 1 to 100.

```
for (int i = 0; i < 2500; i++) {
    long startNanos = System.nanoTime();
    query1.apply(entityManager);
    timer.update(System.nanoTime() - startNanos, TimeUnit.NANOSECONDS);

    startNanos = System.nanoTime();
    query2.apply(entityManager);
    timer.update(System.nanoTime() - startNanos, TimeUnit.NANOSECONDS);
}
```

When executing our test case using two JPQL queries, we get the following results based on the underlying plan cache size:

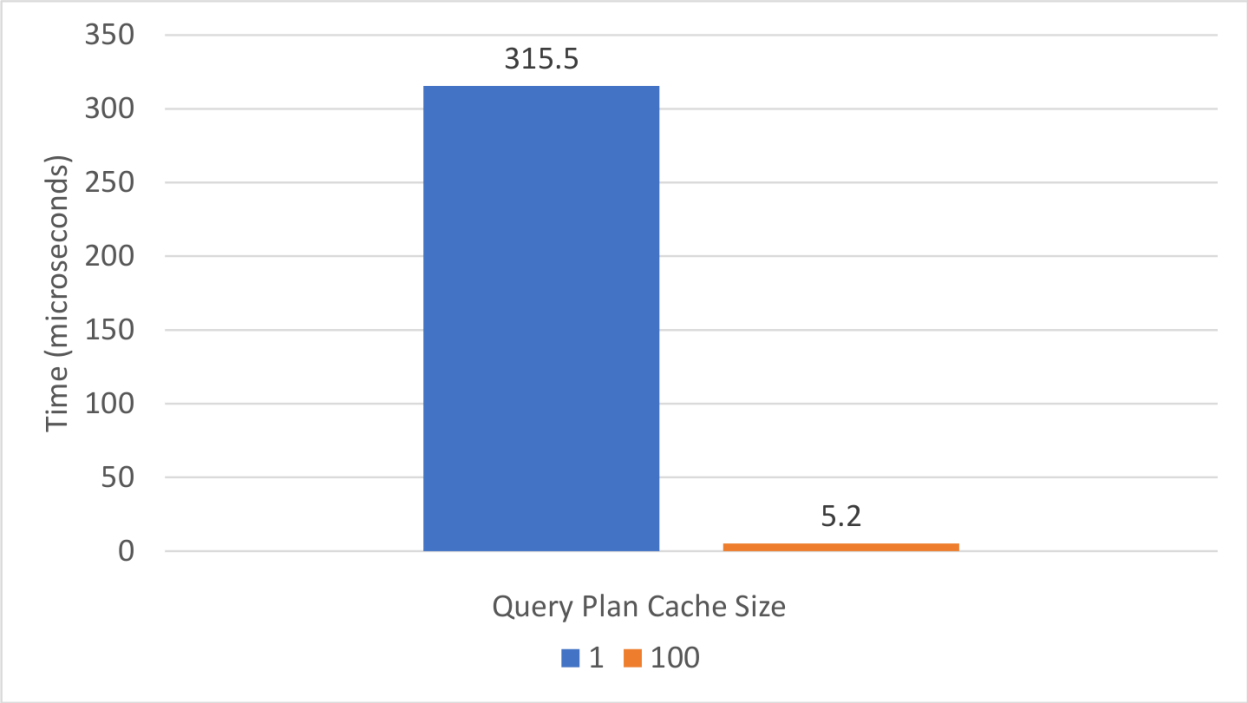

Figure 14.6: Entity query plan cache performance gain

When the plan cache size is 100, both entity queries will be compiled just once since any successive query execution will fetch the previously compiled plan from the cache, therefore speeding up the `Query` object creation.

On the other hand, when the query plan cache size is 1, the entity queries are compiled on every execution, hence the Query‘ object creation will take way longer this time.

Only for entity queries, the plan cache can really make a difference in terms of performance. For native queries, the gain is less significant.

For entity queries (JPQL and Criteria API), it is important to set the `hibernate.query.plan_cache_max_size` property so that it can accommodate all queries being executed. Otherwise, some entity queries might have to be recompiled, therefore increasing the transaction response time.

For native SQL queries, the `ParameterMetadata` cache can also provide a performance improvement, although not as significant as for entity queries:

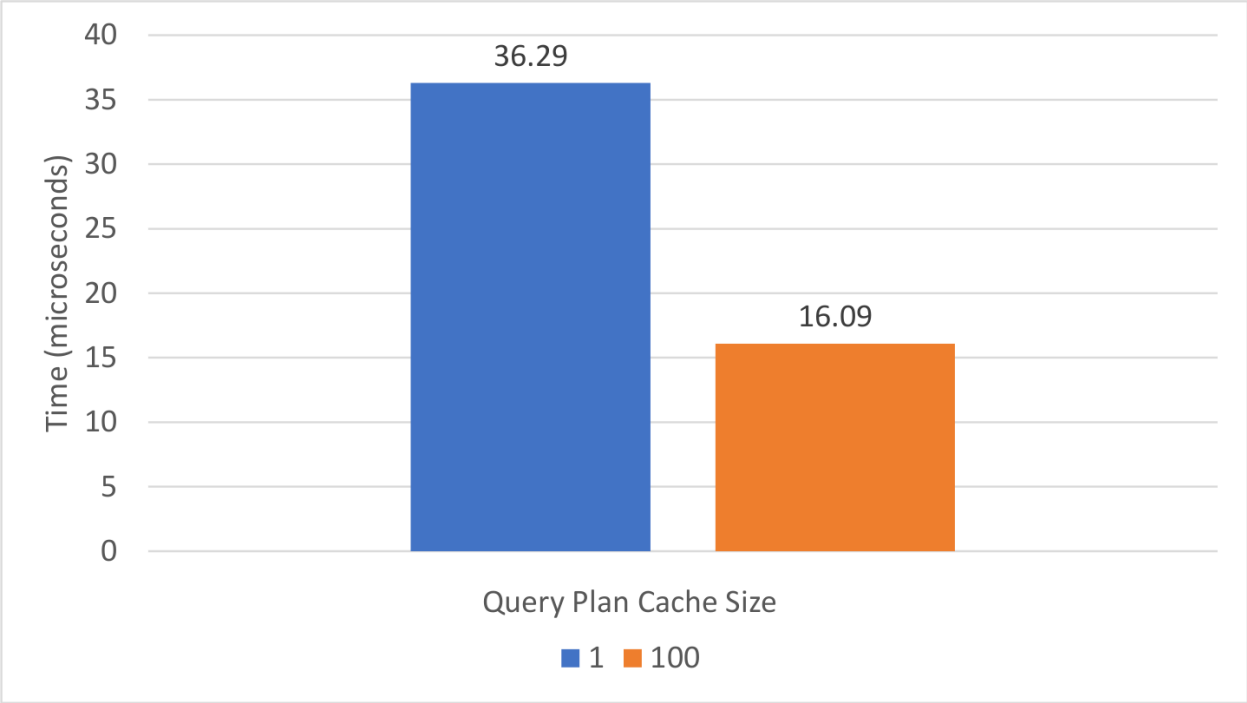

Figure 14.7: Native SQL query plan cache performance gain

15. Caching

15.1 Caching flavors

Caching is everywhere. For instance, the CPU has several caching layers to decrease the latency associated with accessing data from the main memory. Being close to the processing unit, the CPU cache is very fast. However, compared to the main memory, the CPU cache is very small and can only store frequently-accessed data.

To speed up reading and writing to the underlying disk drive, the operating system uses caching as well. Data is read in pages that are cached into main memory, so frequently-accessed data is served from OS buffers rather than the disk drive. Disk cache improves the write operations as well because modifications can be buffered and flushed at once, therefore improving write throughput.

Since indexes and data blocks are better off served from memory, most relational database systems employ an internal caching layer.

So even without explicitly setting up a caching solution, an enterprise application already uses several caching layers. Nevertheless, enterprise caching is most often a necessity, and there are several solutions that can be used for this purpose.

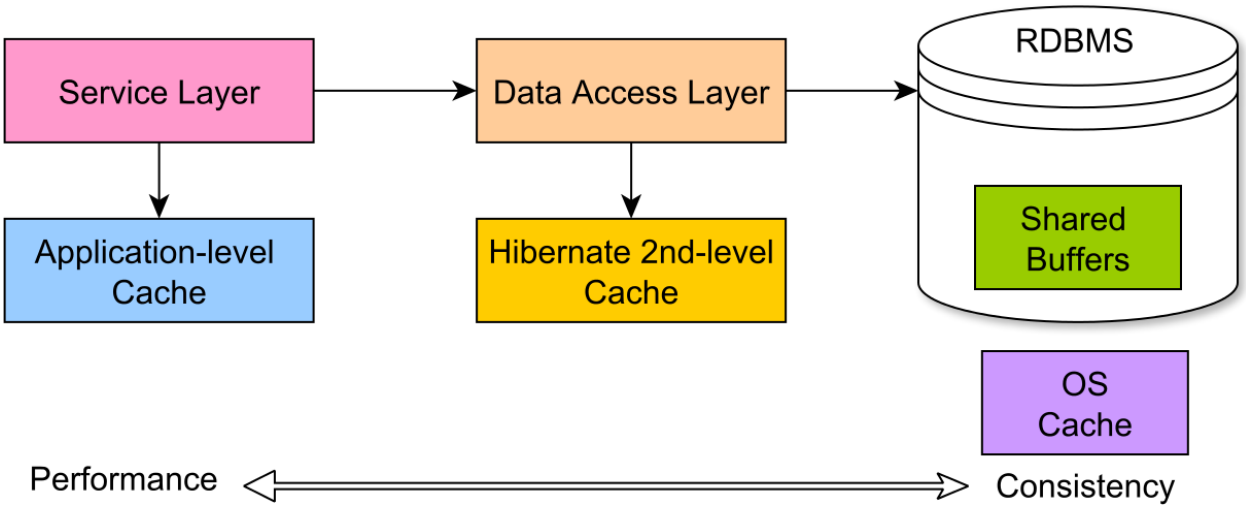

Figure 15.1: Enterprise caching layers

As illustrated in the diagram above, caching entails a trade-off. On one hand, bypassing the underlying data access layer can speed up reads and writes. However, the farther the caching solution is situated, the more difficult it is for it to maintain consistency with the underlying database system.

For it guarantees ACID transactions, the database cache is highly consistent, so, from a data integrity perspective, it entails no risk of reading stale data. However, the database engine can only spare disk access, and so it cannot alleviate the networking overhead. More, if the data access layer needs to fetch an aggregate that spans over multiple database tables, the result set would either contain many joins, or it will require multiple secondary queries. The more complex the data access pattern, the more work a database server has to do, and, for this reason, it is common to use an application-level cache as well.

Most often, application-level caches are key-value stores. Once an aggregate is fetched from the database, it can be stored in the application cache so that any successive request can bypass the database entirely. The application-level cache can outperform the database engine because it can bypass the networking overhead associated with fetching result sets.

Another very important reason for using an application-level caching solution is that it can provide a safety hook for when the database has to be taken down for maintenance.

If the front-end cache stores a sufficient amount of data, it can serve as a temporary replacement for the database system, allowing read-only operations to be served from the cache. Even if write operations are prevented while the database system is unavailable, the read-only mode increases the overall system availability.

However, application-level caches come at a price, and ensuring consistent reads is no longer a trivial thing to do. Because of its tight integration with Hibernate, the second-level cache can avoid many consistency-related issues associated with application-level caches.

The Hibernate second-level cache is a data access caching solution that aims to reduce database load when it comes to fetching entities. Along with it collection cache component, the second-level cache allows retrieving an entire entity graph without a single access to the database.

As explained in the previous chapter, fetching entities is usually associated with propagating entity state transitions. Therefore, the second-level cache can improve response time for read-write transactions without compromising data consistency.

Application-level caches are useful for read scenarios, while the second-level cache consistency guarantee is better suited for offloading write traffic.

15.2 Cache synchronization strategies

In database nomenclature, the system of record represents the source of truth when information is scattered among various data providers. Duplicating data, so that it resides closer to application layers, can improve response time at the price of making it more difficult to synchronize the two data copies. To avoid inconsistent reads and data integrity issues, whenever a change occurs in the system, it is very important to synchronize both the database and the cache.

There are various ways to keep the cache and the underlying database in sync, and this section is going to present some of the most common cache synchronization strategies.

15.2.1 Cache-aside

The application code manually manages both the database system and the caching layer.

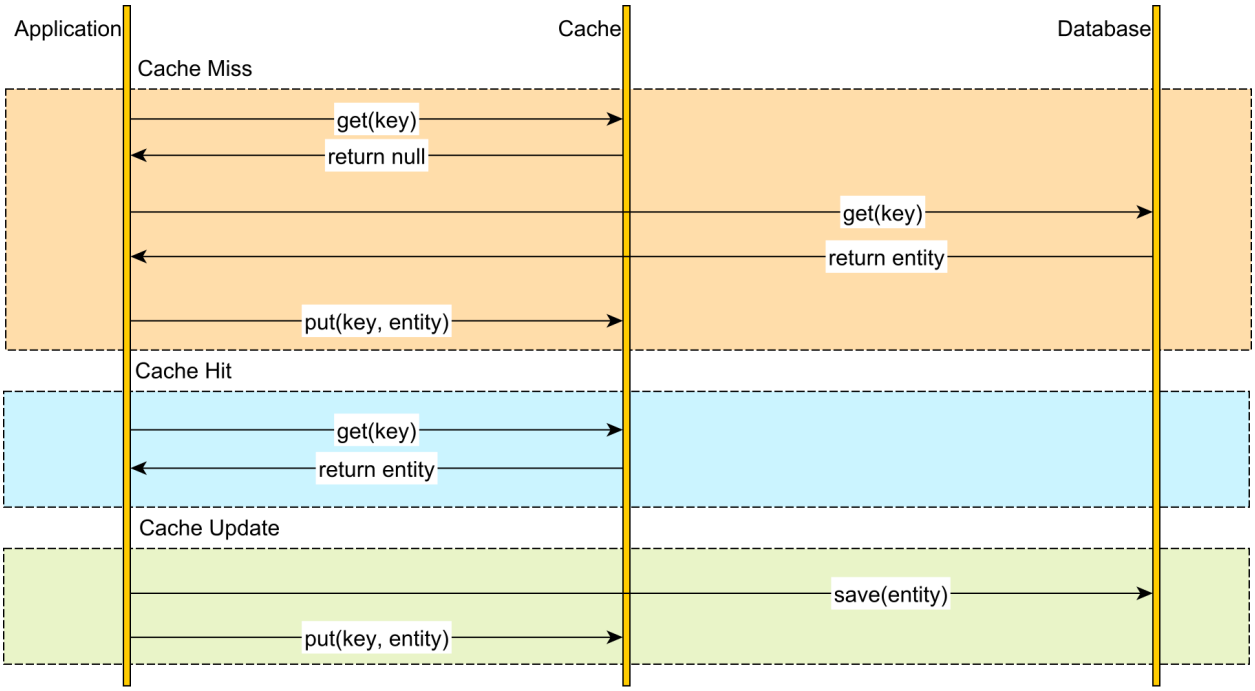

Figure 15.2: Cache-aside synchronization strategy

Before hitting the database, the application logic inspects the cache to see if the requested entity was previously loaded. Whenever an entity changes, the application must update both the database and the cache store.

Mixing application logic with caching management semantics breaks the Single Responsibility Principle. For this reason, it is good practice to move the caching logic into an AOP (aspect-oriented programming) interceptor, therefore decoupling the cache management logic from the business logic code.